Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution

Vishnu [Surname]
Department of Computer Science and Engineering
Saveetha Engineering College, Chennai, India
email@example.com

[Co-author 2]
[Department, Institution]
email@example.com

[Co-author 3]
[Department, Institution]
email@example.com

***Abstract*—Online judge systems have largely been designed around single-file algorithmic problems that are graded by comparing standard output against expected output. Programming education and technical assessment increasingly require learners to work on repository-level codebases, such as fixing a defect in a Django or Node.js project and passing its test suite. Executing such submissions is expensive because every submission needs a prepared runtime with dependencies, and starting a fresh container for each submission introduces cold-start latency that grows painful under concurrent load. This paper presents Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution, a sandboxed execution service that combines an asynchronous Redis queue, a decoupled pool of always-active workers, and a pool of pre-warmed Docker containers organized per runtime image. Instead of requesting an exact container configuration, a submission requests a generic framework runtime, which raises the reuse rate of idle containers. A pool daemon maintains a target idle buffer, reconciles stale state, and removes orphaned containers. Submissions are idempotent, ownership-checked, and executed in network-disabled, resource-limited containers using standard test runners such as pytest and jest. Only the execution layer is scaled, and the API layer remains stateless. We describe the architecture, the measurement methodology used to compare the warm-pooled design with an on-demand container baseline, and the limitations and planned extensions of the current implementation.**

***Index Terms*—online judge, container warm pool, cold start, repository-level assessment, sandboxed code execution, Docker, Redis**

I. Introduction

Automated assessment of programming work removes the need for students to install compilers and IDEs locally and relieves instructors of manually grading every submission. Online code execution systems such as Judge0 move the heavy lifting to a remote, sandboxed environment and expose it through a web API [1]. This model is particularly valuable for remote and online learning [2].

Most existing systems, however, assume the algorithmic contest model in which a single source file reads standard input and writes standard output, and a judge compares the output with the expected output. This model cannot express many realistic software engineering tasks. A debugging challenge drawn from production practice involves a multi-file repository, a framework, installed dependencies, and a test suite that must pass after the learner has fixed a defect. We call this class of task *repository-level debugging challenges*.

Supporting such challenges creates a performance problem. Preparing an environment that already contains the framework and its dependencies is slow, and doing so for every submission means that container initialization dominates the latency of short-lived executions, a bottleneck also identified in the serverless literature [3]. The problem is amplified when many learners submit at the same time, for example at the end of a timed exercise.

This paper presents *Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution*, a code execution service built for repository-level debugging challenges. Its contributions are as follows.

1) A **framework-native assessment** model in which a submission is a set of files executed against a test suite with an industry-standard runner, in place of standard input and output matching.
2) A **decoupled warm container pool** in which pre-provisioned containers per runtime image are tracked in Redis idle and busy sets and maintained by a pool daemon, so that submissions avoid on-demand container start-up.
3) A **generic-runtime request policy** that reuses a container for any submission targeting the same framework runtime, rather than requiring an exact configuration match.
4) A **reliable asynchronous pipeline** with client-supplied idempotency keys, job ownership checks, and an optional completion callback.
5) An **evaluation methodology** for comparing warm-pooled execution against an on-demand container baseline under concurrent load.

The remainder of the paper is organized as follows. Section II reviews related work and the gaps that motivate our design. Section III describes the architecture. Section IV

describes the experimental methodology. Section V discusses limitations, and Section VI outlines planned extensions. Section VII concludes.

II. Related Work

A. Online Code Execution Systems

Došilović and Mekterović describe Judge0, a robust and scalable online code execution system [1]. It separates the web API from the execution environment with an asynchronous Redis queue. Code execution engines pull submissions from the queue, and each submission runs in a fresh Isolate sandbox. The API returns a job identifier and the client subsequently polls for the result. Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution adopts the same separation between API and execution and the same asynchronous job lifecycle. Two aspects of Judge0 motivate our work. First, the authors note that the optimal number of execution engines with respect to machine resources and compiler or interpreter type remains an open question. Second, Judge0 accepts an arbitrary standard input and reads the standard output, which restricts it to single-file algorithmic challenges and prevents execution of repository-level codebases. Judge0 also creates a fresh sandbox per submission, which is inexpensive for a single source file but not for a prepared framework environment. Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution instead keeps pre-warmed containers and reuses them after cleaning.

B. Online Judges for Programming Education

Kim et al. present a web-based online judge for programming education that removes the need for local compilers and automates grading, and that is built from open-source components [2]. Like Judge0, the system targets single-file algorithmic problems and contests. Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution shares its educational motivation and open-source stack, but supports multi-file codebases and framework test suites.

C. Container Reuse for Cold-Start Mitigation

Suo et al. identify container initialization as the primary bottleneck for short-lived serverless executions and propose HotC, a framework that maintains a live container runtime pool [3]. Two limitations of HotC are relevant to our design. First, reuse requires an exact configuration match. For example, a pooled container with Python 3.11 and packages A and B is rejected for a request that additionally needs package C, even though the two are nearly identical, and a new container is started. Second, tracking containers across many machines becomes difficult, and the authors propose a distributed key-value store as future work. Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution requests a generic container capable of running a given framework codebase, for example a Django runtime, instead of requesting an exact configuration, and uses Redis, a key-value store, to coordinate idle and busy container sets for fast lookup.

TABLE I
Summary of Gaps Addressed by Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution

Work	Gap	Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution
[1]	Single-file, stdin/stdout only; fresh sandbox per job	Multi-file repositories; reused warm containers
[2]	Single-file algorithmic problems	Framework-level challenges
[3]	Exact-configuration reuse; tracking at scale	Generic runtime request; Redis-tracked pools
[4]	Per-run container lifecycle; client-side restrictions	Warm pool; server-side enforcement
[5]	Whole-application cloning; 5–12% overhead; static user assignment	Execution-layer scaling; stateless API

D. Docker-Based Online Judges

Several systems use Docker to provide isolated environments for executing student code and support multi-file tasks [4]. The reviewed system creates and destroys a container for every execution, which produces cold-start latency under high concurrent load. It targets introductory language courses such as C++, Java, and Python, and relies on client-side browser restrictions such as disabling paste, which can be bypassed with browser developer tools. Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution targets ticket-style, full-stack framework challenges that resemble production software engineering tasks, uses established test runners, maintains a warm pool per framework, and enforces restrictions on the server side.

E. Distributed Online Judges

MetaOJ addresses the performance bottleneck of massive numbers of concurrent users by cloning the entire judge application into multiple instances managed by a master manager [5]. Synchronizing scoring information back to the manager costs between 5% and 12% of performance under normal high workloads, and users are statically assigned to servers by geography. Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution scales only the execution layer. Its API layer is stateless, so any client can reach the single API endpoint, and the Redis queue distributes work to the next available worker and warm container.

III. System Design

A. Overview

Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution consists of a FastAPI gateway, a Redis instance, a set of RQ workers, a pool daemon, the Docker engine hosting sandbox containers, and a React/Vite administration console. Fig. 1 shows the architecture. The design follows four principles: (i) the web-facing API is decoupled from execution through a queue, (ii) containers are prepared before they are needed, (iii) all submitted code is treated as untrusted, and (iv) every piece of operational state needed for coordination lives in Redis, so that the API remains stateless.

B. Asynchronous Execution API

A client submits files, a runtime, and a test command with `POST /v1/execute`. The gateway validates the request, enqueues a job on the `submission_queue` in Redis, and immediately returns a job identifier. The client then retrieves the job state (pending, running, completed, or failed) and its result through a status endpoint, following the asynchronous pattern used by Judge0 [1]. The current implementation uses bounded polling. An optional `callback_url` allows the service to send the completion or failure result to the client asynchronously.

Idempotency. Each request carries a client-supplied UUID idempotency key. Redis stores an idempotency lease with a 48-hour time to live, so a retried request does not trigger a second, expensive execution.

Ownership. Each job is associated with the identity of the submitting API client. Status retrieval verifies this association so that one client cannot read another client's job.

C. Redis-Coordinated State

Redis holds the job queues, job status and results, idempotency records, job ownership, pool configuration, and the sets of idle and busy container identifiers for each runtime image. Because all coordination state is external to the API processes, any API instance can accept any request, and the load is distributed to the next available worker by the queue. This contrasts with approaches that replicate the entire application and must synchronize state between instances [5].

D. Decoupled Warm Container Pool

Workers are always active and continuously pull jobs from the queue, as in Judge0 [1]. Unlike Judge0, a worker does not create a new sandbox per submission, because starting a container that holds a full codebase environment is slow and expensive. Instead, Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution keeps a configurable number of pre-warmed containers for each runtime image, an approach in the spirit of runtime pooling in HotC [3].

Pool definitions. Each pool is defined by an image tag, a maximum container count, a default-pool flag, and timestamps.

Acquire and release. For each submission, the worker takes an idle container identifier from the Redis idle set of the required image, marks it busy, executes the submission, cleans the workspace, and returns the container to the idle set. Tracking idle and busy sets in Redis gives constant-time lookup and avoids contention on a central scheduler.

Pool daemon. A background daemon periodically checks each pool and provisions containers to maintain a target idle buffer, so that a burst of submissions does not wait for container creation. When the idle population exceeds the configured limit, the daemon removes excess idle containers. A reconciliation step removes stale identifiers from Redis and removes Docker containers that are no longer represented in pool state, which clears orphaned containers. On shutdown, the daemon attempts to terminate managed containers and clear pool state.

Generic runtime requests. A submission asks for a runtime capable of running its framework, such as a Django or Node.js runtime, rather than for a container with an exact configuration. This avoids the rejection of nearly identical containers observed under exact-match reuse [3] and raises the fraction of submissions served by an already-warm container.

E. Runtime Image Management

Administrators define runtime images for Python and Node.js, search PyPI and npm for packages and versions, and trigger asynchronous builds on a dedicated `image_builds` queue. Runtime and dependency definitions are normalized, and the image tag is derived deterministically from a content hash, so an image that already exists locally is not rebuilt. Image definitions are stored with atomic file replacement to avoid partially written files, and build logs are retained for inspection.

F. Sandboxed Execution

For each submission, the worker packs the submitted files into an in-memory tar archive and streams them into `/workspace` of the acquired container, using a base64 pipe where required to work around archive-copy edge cases with a read-only root filesystem. The test command is then run inside the container with the following controls: non-root execution, disabled networking, memory and process-count limits, a read-only root filesystem, and temporary writable locations. Standard test runners such as pytest and jest produce the results, which allows detailed handling of individual test cases in place of whole-output comparison. After execution, the workspace is cleaned and the container returns to the idle set. These controls follow the general requirement of sandboxing and resource limiting for untrusted code in online execution systems [1]. They are defence in depth and are not by themselves treated as a complete security boundary.

Telemetry. Each job records workspace creation time, execution time, cleanup time, and total worker time, which provide the measurements used in Section IV.

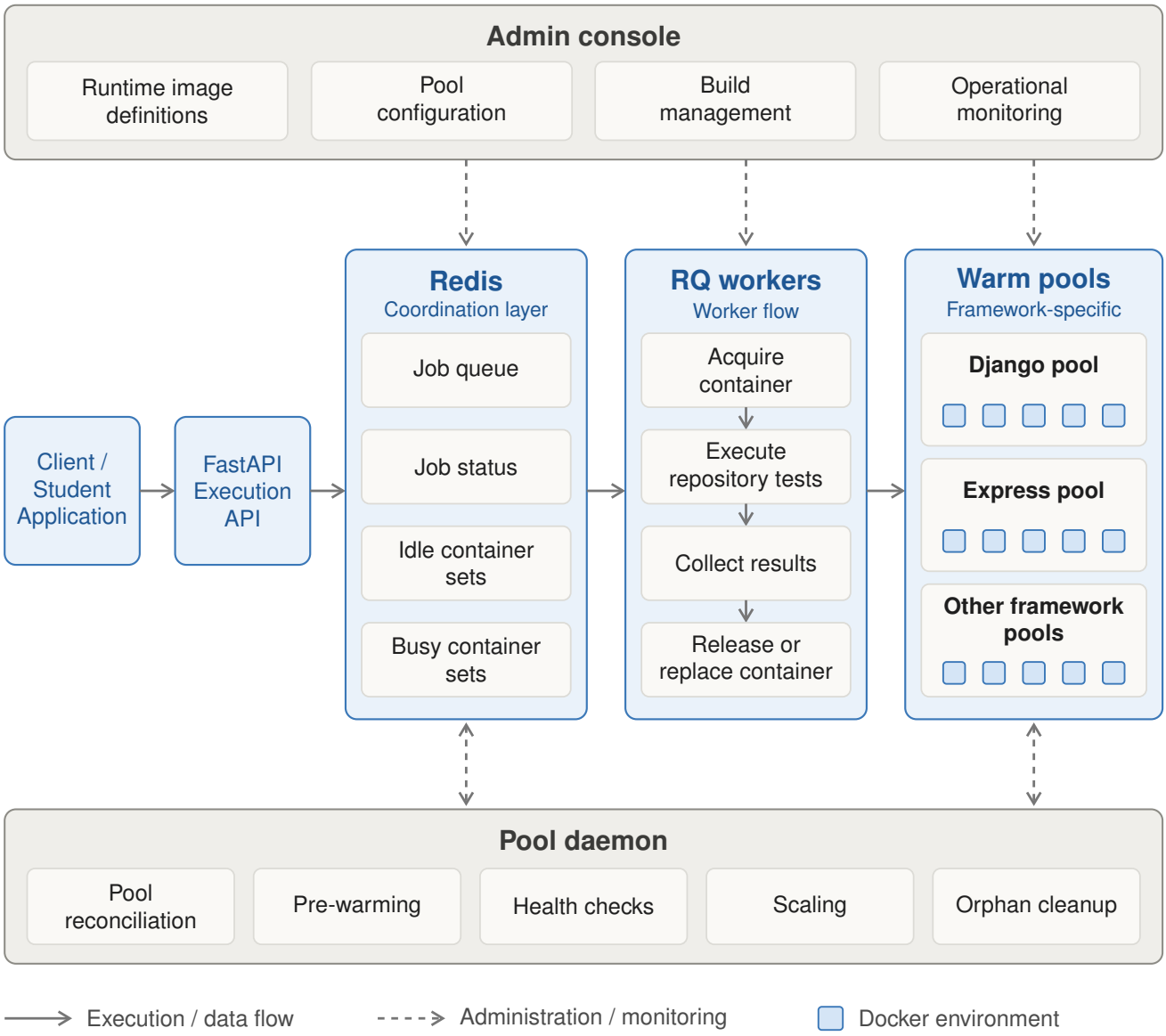


Fig. 1. Proposed Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution architecture. Clients submit code to the FastAPI backend, which enqueues a job in Redis and returns a job identifier. Workers dequeue jobs, acquire an idle container for the required runtime, execute the submission, store the result in Redis, and release the container. The pool daemon maintains the idle buffer of each pool.

G. Administration Console

A React/Vite console lets administrators manage runtime images, package selection, Dockerfile previews, build logs, and warm pools, including free, occupied, total, and maximum capacity. Administrator passwords are verified with bcrypt against a configured hash. Login attempts are rate limited by IP address with a temporary lockout, and session tokens generated from a cryptographically secure source are sent in an `HttpOnly`, `SameSite=Strict` cookie. Administrative and public execution routes use separate CORS policies, and responses carry `X-Content-Type-Options: nosniff` and `X-Frame-Options: DENY` headers.

IV. Evaluation Methodology

This section reports the observed performance of the running Patchwork-Service implementation under nine 100-request concurrency tests. The experiments examine worker-count scaling and changes to container-pool configuration. They do not include a measured per-submission on-demand-container baseline; consequently, the results characterize the tested warm-pool deployment and its operational limits rather than quantify a causal latency reduction relative to cold container creation.

A. Experimental Setup

Execution host and load generator. The service and load generator ran on separate machines. Table II summarizes the

hardware and software details available from the benchmark record. Exact server CPU model, Docker version, storage details, and the Redis/RQ versions were not recorded in that report and are therefore not inferred here.

Workload and procedure. Each of the nine tests issued 100 concurrent requests and used a 180-s grace period. The API request and callback workflow was kept generally consistent across the sequence. The first five tests varied the RQ worker count over $\{1, 2, 4, 8, 16\}$ while retaining a maximum pool size of 10 containers. Tests 6–9 used a configured maximum pool size of 20 and compared worker counts of 16, 10, 8, and 6, respectively. The supplied benchmark record describes one run for each configuration; repeated trials and confidence intervals were not available.

The settings in Table III are the configured values reported for each group; they are not proof that every live process had loaded the corresponding values. In particular, Test 6 recorded container-acquisition failures after approximately 10.2–10.4 s despite the configured 30-s timeout. The benchmark notes that workers had not been restarted after the configuration change, so stale worker configuration is a plausible explanation, but the available evidence does not establish it as the only cause.

B. Metrics

The dashboard-reported throughput is expressed in requests per second. We also report total test elapsed time and end-to-end completion latency, measured from request submission until the callback response was received. Mean, median, and 95th-percentile (P95) latency are included where recorded. End-to-end latency includes queueing, container acquisition, workspace preparation, execution, cleanup, and callback delivery; it is not equivalent to code execution time. For Test 6, the detailed export additionally reported queue delay and worker timing. Failure observations are reported conservatively: the phrase "no failure banner shown" is not interpreted as proof of zero failed executions, and missing dashboard failure counts are not treated as zero.

C. Results

Table IV summarizes all nine tests in the order in which the experiments were reported. Pool maxima refer to configured maximum container counts. The failure column records only what was visible in the supplied dashboard or detailed export.

D. Discussion

Worker-count scaling with the original pool. As shown in Fig. 2, the first four tests increased dashboard throughput from 0.655 requests/s with one worker to 2.314 requests/s with eight workers. Over the same comparison, mean end-to-end latency decreased from 80.05 s to 20.68 s, a reduction of approximately 74.2%. This indicates that additional workers substantially improved aggregate processing capacity over the lower worker counts in this workload. Increasing to 16 workers produced the highest observed raw throughput, 2.645 requests/s, and the lowest observed mean latency in the original-pool group (18.35 s), but six container-acquisition failures were reported. Thus, the Test 5 result should not be interpreted as the best reliable operating point solely on the basis of throughput.

Tests with the updated pool configuration. Among Tests 6–9, the eight-worker configuration (Test 8) had the highest dashboard throughput (2.247 requests/s) and the lowest mean and P95 latency (22.17 s and 41.75 s, respectively). It outperformed the ten-worker run (Test 7: 1.957 requests/s, 31.14-s mean, 50.40-s P95) and the six-worker run (Test 9: 2.039 requests/s, 23.39-s mean, 47.81-s P95) on these dashboard-level measures. However, the separate failure count was not visible for Tests 7–9; the evidence therefore supports describing eight workers as the best observed performance balance in this subset, not as a validated failure-free configuration.

Container acquisition and queueing. Tests 5 and 6 each reported six failures associated with inability to acquire a warm container. The detailed Test 6 export recorded approximately 28.612 s of mean queue delay and 36.034 s of mean end-to-end completion time, so queue delay represented about 79.4% of the latter. The six failed submissions, however, had queue delays of only approximately 9–14 ms and failed after about 10.2–10.4 s with a container-acquisition error. This distinction indicates that queue waiting for successful jobs and container-acquisition failure were separate observed failure/performance paths in Test 6. The export also reported mean execution time of about 4.51 s, workspace creation of 0.64 s, cleanup of 0.68 s, and total worker time of about 6.86 s; these fields represent different timing intervals and should not be summed to reconstruct end-to-end latency.

Interpretation and threats to validity. The diminishing and inconsistent benefit at higher worker counts is consistent with contention for host resources and with a mismatch between worker concurrency and warm-container supply. Nevertheless, CPU utilization, per-core saturation, memory pressure, swap activity, disk I/O, and time-series pool occupancy were not collected. CPU saturation is therefore a plausible hypothesis rather than a directly measured cause. The experiment also changed multiple pool parameters between test groups, did not confirm that every worker loaded the updated timeout, used one reported run per configuration, and did not expose failure counts for three later tests. Finally, the available data do not include an on-demand-container baseline, per-configuration pool-hit rates, or correctness results (the dashboard reported correct results as unavailable). These limitations preclude a quantitative claim that warm pooling reduced latency relative to on-demand creation.

E. Provisional Configuration

For the next controlled experiment, the observed results support using eight workers as a provisional starting point with five initial warm containers, a configured pool maximum of 20, provisioning concurrency of five, a 30-s acquisition timeout, framework target buffer of 12, and Redis pool target of five. Before drawing a stable-performance conclusion, all worker processes should be restarted and checked for the active timeout, the number of pool daemons should be verified,

TABLE II
HARDWARE AND SOFTWARE ENVIRONMENT REPORTED FOR THE BENCHMARK

Parameter	Service host	Load generator
Processor	Intel Core i5, 12th generation (exact model not recorded in benchmark report)	Intel Core Duo (exact model not recorded)
Memory	16 GB	12 GB
Operating system	Linux Mint, terminal-only mode	Linux Mint XFCE
API / runtime	FastAPI 0.139.0; Python 3.12.3	Node.js 22.23.3; k6 2.3.0
Queue and execution	Redis / RQ; Docker-managed execution containers	k6-based concurrent request generation

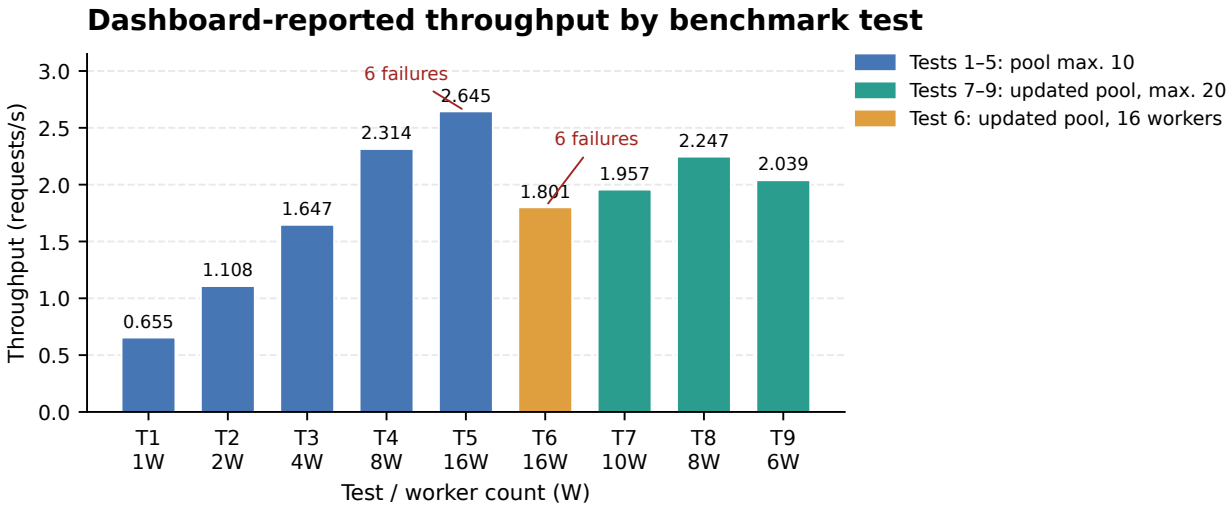


Fig. 2. Dashboard-reported throughput across the nine tests. Tests 1–5 used a maximum pool size of 10; Tests 6–9 used a configured maximum of 20 with other pool settings changed. The two tests with six reported failures are annotated.

TABLE III
CONTAINER-POOL SETTINGS BY TEST GROUP

Setting	Tests 1–5	Tests 6–9
Initial warm containers	5	5
Maximum pool size	10	20
Provisioning concurrency	2	5
Configured acquisition timeout	10 s	30 s
Framework target buffer	5	12
Redis pool target	5	5

and CPU, memory, swap, disk I/O, and idle/busy/provisioning container counts should be captured during the run. The eight-worker test should be repeated at least three times, and successful executions, infrastructure failures, missing callbacks, and correctness outcomes should be reported separately. A direct comparison against a matched on-demand baseline is required to quantify the latency benefit attributable to container reuse.

V. LIMITATIONS

Several limitations apply to the current implementation. Clients retrieve results by polling, which adds request traffic and a delay of up to one polling interval. Warm containers consume memory while idle, so the target buffer trades resource use for latency. Container reuse relies on thorough workspace cleanup, and a container that crashes while marked busy can leave stale state until reconciliation. Administrator sessions are held in application memory and image definitions are stored in files, which is adequate for a single-instance deployment but needs shared storage for replicated deployments. Client API-key authentication is not yet backed by persistent credentials. Finally, network isolation and a read-only root filesystem are useful controls but should not be treated as a complete security boundary, and Judge0 itself notes that the optimal number of execution engines for given resources remains an open question [1], which we have not resolved.

VI. FUTURE WORK

Push-based result delivery. Replace polling with server-sent events or WebSockets so that clients receive status changes as they happen, reducing request overhead and latency.

Worker management. Add an administration page, designed like the existing pool management page, to monitor and configure workers, with heartbeats, queue visibility, enable and disable, and graceful shutdown that does not silently kill in-flight work. Worker control would sit behind a narrow authenticated controller with allowlists, and the console would never accept arbitrary commands.

TABLE IV
RESULTS OF THE NINE 100-REQUEST CONCURRENCY TESTS

Test	Workers	Pool max.	Elapsed (s)	Mean (s)	Median (s)	P95 (s)	Throughput (req/s)	Failure evidence
1	1	10	152.8	80.05	79.84	145.26	0.655	No banner shown
2	2	10	90.4	44.26	43.54	85.86	1.108	No banner shown
3	4	10	60.9	29.17	27.84	57.30	1.647	No banner shown
4	8	10	43.3	20.68	19.41	41.83	2.314	No banner shown
5	16	10	37.9	18.35	16.30	35.66	2.645	6 failures
6	16	20	55.7	36.03	33.28	54.25	1.801	6 failures
7	10	20	51.2	31.14	28.49	50.40	1.957	Count unavailable
8	8	20	44.6	22.17	20.82	41.75	2.247	Count unavailable
9	6	20	49.2	23.39	21.36	47.81	2.039	Count unavailable

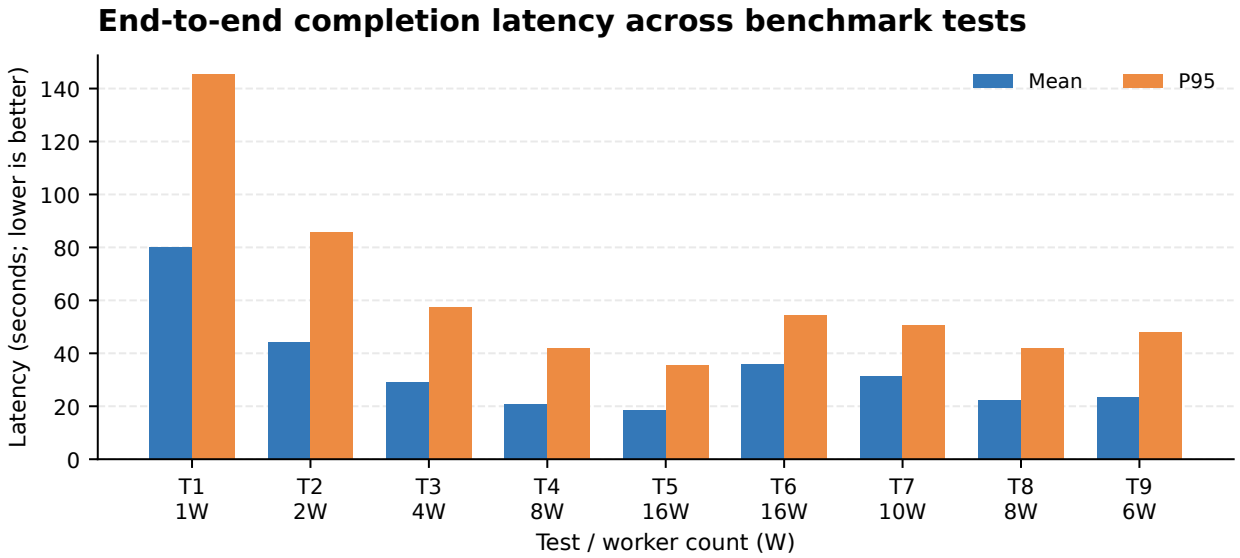


Fig. 3. Mean and P95 end-to-end completion latency for each test. Both measures include asynchronous queueing and callback delivery; lower values indicate shorter observed completion times.

Learning management integration. Develop a Moodle plugin through which instructors configure code-execution activities and students submit code and view results, motivated by education-oriented judge systems that place course and feedback workflows around an execution backend [2].

Reliability and scaling. Add leases and heartbeats for busy-container assignments with timeout-aware recovery, preemptive scaling of the idle buffer based on submission rate, graceful degradation and shutdown of the execution layer, persistent client credentials, and Redis-backed administrator sessions.

Broader evaluation. Extend the study to additional frameworks and multi-host pools.

VII. CONCLUSION

We presented Design and Performance Evaluation of a Warm-Pooled Container Architecture for Scalable Repository Level Debugging Challenge Execution, a sandboxed execution service for repository-level debugging challenges. By combining a Redis-backed asynchronous pipeline, decoupled always-active workers, and per-runtime pools of pre-warmed containers requested by generic framework runtime, the design addresses the single-file limitation of classical online judges and the cold-start cost of per-submission containers, while scaling only the execution layer. The evaluation methodology in this paper defines how the latency benefit of warm pooling is to be quantified under concurrent load. The planned extensions,

including push-based result delivery, worker management, and learning management integration, build on this foundation.

REFERENCES

[1] H. Z. Došilović and I. Mekterović, "Robust and Scalable Online Code Execution System," in *Proc. 43rd Int. Convention on Information, Communication and Electronic Technology (MIPRO)*, 2020, pp. 1627–1632, doi: 10.23919/MIPRO48935.2020.9245310.

[2] S. Kim, J. Park, S. Jeon, and D. S. Seo, "Web-Based Online Judge System for Online Programming Education," in *Proc. IEEE Int. Conf. Consumer Electronics (ICCE)*, 2022, pp. 1–3, doi: 10.1109/ICCE53296.2022.9730422.

[3] K. Suo, J. Son, D. Cheng, W. Chen, and S. Baidya, "Tackling Cold Start of Serverless Applications by Efficient and Adaptive Container Runtime Reusing," in *Proc. IEEE Int. Conf. Cluster Computing (CLUSTER)*, 2021, doi: 10.1109/Cluster48925.2021.00018.

[4] [Authors], "An Experimental Online Judge System Based on Docker Container for Learning and Teaching Assistance," [venue, year, pages, DOI].

[5] [Authors], "MetaOJ: A Massive Distributed Online Judge System," [venue, year, pages, DOI].